BlimBlom – App Development Plan **Tool by tool**

Covers only the app/frontend track (role A, with B and C support). Written Thu 15 Oct, 21:25 — deadline Sun 18 Oct, noon. Less than 3 days left.

> **Architecture decision (make this first):** build **client-only** — the app calls Gemini and Firestore directly from the browser. No Cloud Run, no custom backend server. This cuts a full day of backend work. Trade-off: the Gemini API key lives in the app code, which is fine for a hackathon demo but should be restricted by domain (step 2) and mentioned as a known limitation in the deck.

Tool stack overview

Tool	Used for	Needs billing?
Google AI Studio	Design and test the Gemini prompt; get API key	No
Firebase Console	Firestore (database), Hosting (deploy), Cloud Messaging (push)	No, on Spark plan
Antigravity or VS Code + Claude Code	Write the actual app code	No
GitHub	Shared code, version history	No
Firebase CLI	Deploy the app to a public URL	No
A real phone + Chrome DevTools	Test install, push notifications, and bugs	No

Tonight – Thu 15 Oct, 21:30–late: Prompt + keys

1. Google AI Studio (aistudio.google.com)

- Sign in, click **Get API key → Create API key**, pick the team's GCP project so usage is tracked in one place. Save the key somewhere private, never in a chat or committed file.
- Open a new prompt, attach 3–4 test audio clips (elephant, chainsaw, normal forest) one at a time.
- Write a system prompt that forces structured JSON output, for example:

```
Analyze this audio clip from a forest acoustic sensor.
Return ONLY valid JSON with this shape:
{
  "event_type": "elephant" | "chainsaw" | "vehicle" | "normal",
  "confidence": 0.0-1.0,
  "urgency": "low" | "medium" | "high",
  "reasoning": "short explanation",
  "alert_message_th": "short Thai alert for a ranger",
  "alert_message_en": "short English alert"
}
```

- Test against each clip, adjust wording until elephant and chainsaw are reliably distinguished from normal forest sound. Copy the final prompt text somewhere safe — it goes straight into the app code tomorrow.

> Output tonight: a working Gemini API key, and a prompt that correctly labels at least 3 of 4 test clips.

Fri 16 Oct, morning: Firebase setup

2. Firebase Console (console.firebase.google.com)

- **Add project** → select the existing GCP project (do not create a new one).
- **Firestore Database → Create database** → location `asia-southeast1` → start in **test mode** (open read/write for now, lock it down later if time allows).
- **Project settings → General → Add app → Web (</>)** → name it `blimblom-app` → copy the `firebaseConfig` object shown, you'll paste it into the app code.
- **Project settings → Cloud Messaging → Web Push certificates → Generate key pair** → copy the VAPID key, needed for push notifications.
- **Restrict the Gemini key:** in Google Cloud Console → APIs & Services → Credentials → click the Gemini key → under *Application restrictions* choose *HTTP referrers* → add the Firebase Hosting domain you'll get in step 5 (you can add it now as `*.web.app/*` and `*.firebaseapp.com/*`, or come back to this after first deploy).

> Output: a `firebaseConfig` object and a VAPID key, both pasted into a notes file for the coding step.

Fri 16 Oct, rest of day: Build the app

3. Antigravity (or VS Code + Claude Code / your usual editor)

- Scaffold a simple web app. Fastest path: plain HTML/JS with Firebase and Gemini loaded via CDN script tags — no build step, no bundler, nothing to configure wrong under time pressure.
- **Screen 1 — Alert list:** a live Firestore listener (`onSnapshot`) on the `incidents` collection, newest first, showing event type, urgency, and time.
- **Screen 2 — Incident detail:** tapping an alert shows the audio clip player, the Gemini reasoning text, the location, and buttons (Acknowledge / On the way / Resolved) that update the Firestore document.
- **Gemini call from the browser:** use the `@google/genai` JS SDK (loaded via CDN or npm) with the API key from step 1, sending the audio file as inline data with the prompt from tonight.
- **Write to Firestore from the browser:** use the Firebase JS SDK (`firebase/firestore`), initialized with the `firebaseConfig` from step 2, to write the Gemini result into the `incidents` collection.
- **Simulator:** a small script or page that loops through the downloaded test audio files and calls the same analyze-and-write function, acting as the "sensors." This can be a second simple HTML page, or a button in the app itself ("Simulate sensor event").
- If splitting work: person A builds the two screens and the Firestore read side; person B (already owns the prompt) wires the Gemini call and the simulator, since they know the prompt and data shape best.

> Done tonight: clicking "simulate" creates a new Firestore document, and it shows up in the alert list without a page refresh.

Sat 17 Oct, morning: First deploy

4. GitHub

- `git init`, commit, push to the shared repo under `app/`. Make sure `.env` or any file holding the raw API key is in `.gitignore` — commit config as a template file instead (e.g. `firebaseConfig.example.js`).

5. Firebase CLI

```
npm install -g firebase-tools
firebase login
firebase init hosting   # select the existing project, public dir = app folder
firebase deploy --only hosting
```

- This gives a public URL like `blimblom.web.app`. Open it in a private/incognito window to confirm it works logged out.
- Go back to step 2's key restriction and add this exact domain if you skipped it earlier.

> Done when: the public URL loads the app and shows live Firestore data, from a device that never touched your laptop.

Sat 17 Oct, afternoon/evening: Push notifications + polish

Firebase Cloud Messaging (code, in the app)

- Register a service worker (`firebase-messaging-sw.js`) that handles background push.
- Request notification permission from the user on first load.
- Get the device's FCM token, save it to a `subscribers` collection in Firestore.
- Simplest trigger for a hackathon: when the app writes a high-urgency incident, also call the FCM REST API directly from the client with the saved token(s) — skip Cloud Functions entirely to stay backend-free. This is not ideal for production but is fine for a demo.

6. Real phone + Chrome DevTools

- On a phone, open the public URL in Chrome, tap **Add to Home Screen**, open it from there once so it registers as an installed PWA.
- Accept the notification permission prompt.
- From a laptop, run the simulator to create a high-urgency incident and confirm a push notification appears on the phone within a few seconds.
- Use Chrome DevTools → Application tab → Service Workers, and the Console, to debug anything that doesn't fire.

> Feature freeze tonight. Done when: a simulated elephant alert makes a real push notification pop up on a real phone, end to end, on the public URL.

Sun 18 Oct, morning: Final check only

- No new features. Only fix bugs that appear on camera during the demo recording (shared with the rest of the team's video/deck work).
- Reload the public URL on a phone that has never opened it before, confirm install + first alert still work.
- Make sure Firestore security rules are not left in test mode without an expiry warning — if still open, that's acceptable for a hackathon demo but mention it as a known limitation if asked.

If behind schedule — cut in this order

1. Acknowledge / On the way / Resolved buttons — a read-only alert list is enough
2. Push notifications — the live alert list alone still demonstrates the core idea
3. Map or location names — plain list is enough
4. A second event type — demo only elephant detection, mention chainsaw detection as designed-for-but-not-shown

Never cut: Gemini analysis working on real audio, a public Firebase Hosting URL, at least one incident flowing end to end into the alert list.

BlimBlom – แผนพัฒนาแอป ไล่ทีละเครื่องมือ

ครอบคลุมเฉพาะสายงานแอป/หน้าบ้าน (คน A โดยมีคน B และ C ช่วย) เขียนเมื่อคืนวันพฤหัส 15 ต.ค. 21:25 น. กำหนดส่งอาทิตย์ 18 ต.ค. เที่ยง เหลือเวลาไม่ถึง 3 วัน

> **ตัดสินใจเรื่องสถาปัตยกรรมก่อนเป็นอันดับแรก:** สร้างแบบ **client-only** คือให้แอปเรียก Gemini และ Firestore ตรงจากเบราว์เซอร์เลย ไม่ต้องมี Cloud Run หรือ backend server แยก วิธีนี้ตัดงานฝั่ง backend ออกไปได้เกือบทั้งวัน ข้อเสียคือ Gemini API key จะอยู่ในโค้ดฝั่งแอป ซึ่งรับได้สำหรับเดโมแข่งขัน แต่ควรจำกัดการใช้งานด้วย domain (ขั้นตอนที่ 2) และพูดถึงเป็นข้อจำกัดที่รู้อยู่แล้วในสไลด์

ภาพรวมเครื่องมือที่ใช้

เครื่องมือ	ใช้ทำอะไร	ต้องผูกบัตรไหม
Google AI Studio	ออกแบบและทดสอบ prompt ของ Gemini, ขอ API key	ไม่ต้อง
Firebase Console	Firestore (ฐานข้อมูล), Hosting (deploy), Cloud Messaging (push)	ไม่ต้อง (แผน Spark)
Antigravity หรือ VS Code + Claude Code	เขียนโค้ดแอปจริง	ไม่ต้อง
GitHub	เก็บโค้ดร่วมกัน, ประวัติการแก้ไข	ไม่ต้อง
Firebase CLI	deploy แอปขึ้น URL สาธารณะ	ไม่ต้อง
มือถือจริง + Chrome DevTools	ทดสอบการติดตั้ง, push notification, และบั๊ก	ไม่ต้อง

คืนนี้ – พฤหัส 15 ต.ค. 21:30 เป็นต้นไป: Prompt + API key

1. **Google AI Studio** (aistudio.google.com)

- ล็อกอิน กด **Get API key → Create API key** เลือกโปรเจกต์ GCP ของทีมเพื่อให้การใช้งานรวมอยู่ที่เดียว เก็บ key ไว้ในที่ส่วนตัว ห้ามวางในแชตหรือ commit ขึ้น GitHub
- เปิดหน้า prompt ใหม่ แนบไฟล์เสียงทดสอบ 3–4 คลิป (ช้าง, เลื่อยยนต์, เสียงป่าปกติ) ทีละคลิป
- เขียน system prompt ให้บังคับตอบเป็น JSON มีโครงสร้าง เช่น:

```
วิเคราะห์ไฟล์เสียงจากเซนเซอร์ในป่านี้
ตอบเป็น JSON เท่านั้น ตามรูปแบบนี้:
{
  "event_type": "elephant" | "chainsaw" | "vehicle" | "normal",
  "confidence": 0.0-1.0,
  "urgency": "low" | "medium" | "high",
  "reasoning": "เหตุผลสั้น ๆ",
  "alert_message_th": "ข้อความแจ้งเตือนภาษาไทยสั้น ๆ สำหรับเจ้าหน้าที่",
  "alert_message_en": "ข้อความแจ้งเตือนภาษาอังกฤษสั้น ๆ"
}
```

- ทดสอบกับทุกคลิป ปรับคำใน prompt จนแยกเสียงช้างและเลื่อยยนต์ออกจากเสียงป่าปกติได้แม่นยำ คัดลอก prompt ฉบับสุดท้ายเก็บไว้ที่ปลอดภัย พรุ่งนี้จะเอาไปใส่ในโค้ดแอปตรง ๆ

> สิ่งที่ต้องได้คืนนี้: Gemini API key ที่ใช้งานได้ และ prompt ที่แยกคลิปทดสอบได้ถูกต้องอย่างน้อย 3 จาก 4 คลิป

ศุกร์ 16 ต.ค. ช่วงเช้า: ตั้งค่า Firebase

2. **Firebase Console** (console.firebase.google.com)

- **Add project** → เลือกโปรเจกต์ GCP ที่มีอยู่แล้ว (อย่าสร้างใหม่)
- **Firestore Database → Create database** → location `asia-southeast1` → เริ่มแบบ **test mode** ไปก่อน (เปิดอ่านเขียนได้ชั่วคราว ค่อยปิดทีหลังถ้ามีเวลา)
- **Project settings → General → Add app → เว็บ (</>)** → ตั้งชื่อ `blimblom-app` → คัดลอกชุด `firebaseConfig` ที่ได้มา จะเอาไปวางในโค้ดแอป
- **Project settings → Cloud Messaging → Web Push certificates → Generate key pair** → คัดลอก VAPID key ไว้ ใช้สำหรับ push notification
- **จำกัดการใช้ Gemini key:** ไปที่ Google Cloud Console → APIs & Services → Credentials → กดที่ Gemini key → ใต้ *Application restrictions* เลือก *HTTP referrers* → เพิ่ม domain ของ Firebase Hosting ที่จะได้ในขั้นตอนที่ 5 (ใส่ไว้ก่อนเป็น `*.web.app/*` และ `*.firebaseapp.com/*` หรือกลับมาทำหลัง deploy ครั้งแรกก็ได้)

> สิ่งที่ต้องได้: ชุด `firebaseConfig` และ VAPID key บันทึกไว้ในไฟล์โน้ตสำหรับขั้นตอนเขียนโค้ด

ศุกร์ 16 ต.ค. ช่วงที่เหลือของวัน: เขียนแอป

3. **Antigravity (หรือ VS Code + Claude Code / เครื่องมือที่ถนัด)**

- สร้างโครงเว็บแอปแบบง่ายที่สุด: ใช้ HTML/JS ธรรมดา โหลด Firebase กับ Gemini ผ่าน script tag จาก CDN ไม่ต้องมี build step หรือ bundler ลดความเสี่ยงตั้งค่าผิดตอนเวลาจำกัด
- **หน้าจอ 1 — รายการแจ้งเตือน:** ใช้ Firestore live listener (`onSnapshot`) กับ collection `incidents` เรียงใหม่สุดก่อน แสดงประเภทเหตุการณ์ ความเร่งด่วน และเวลา
- **หน้าจอ 2 — รายละเอียดเหตุการณ์:** กดที่รายการแจ้งเตือนแล้วแสดงตัวเล่นไฟล์เสียง ข้อความวิเคราะห์จาก Gemini ตำแหน่ง และปุ่ม รับเรื่อง / กำลังไป / ปิดเหตุ ที่แก้ไขเอกสารใน Firestore ได้
- **เรียก Gemini จากเบราว์เซอร์:** ใช้ JS SDK `@google/genai` (โหลดผ่าน CDN หรือ npm) พร้อม API key จากขั้นตอนที่ 1 ส่งไฟล์เสียงเป็น inline data พร้อม prompt ที่ทำไว้เมื่อคืน
- **เขียนลง Firestore จากเบราว์เซอร์:** ใช้ Firebase JS SDK (`firebase/firestore`) ที่ตั้งค่าด้วย `firebaseConfig` จากขั้นตอนที่ 2 เพื่อบันทึกผลจาก Gemini ลง collection `incidents`
- **สคริปต์จำลองเซนเซอร์:** ทำสคริปต์หรือหน้าเล็ก ๆ ที่วนเล่นไฟล์เสียงทดสอบที่ดาวน์โหลดไว้ แล้วเรียกฟังก์ชันวิเคราะห์และบันทึกเดียวกัน ทำหน้าที่เป็น "เซนเซอร์" อาจทำเป็นหน้า HTML ที่สอง หรือปุ่มในแอปเอง ("จำลองเหตุการณ์เซนเซอร์")
- ถ้าแบ่งงาน: คน A ทำสองหน้าจอและฝั่งอ่านข้อมูลจาก Firestore ส่วนคน B (ที่ดูแล prompt อยู่แล้ว) ต่อสาย Gemini กับสคริปต์จำลอง เพราะรู้จัก prompt และรูปแบบข้อมูลดีที่สุด

> เป้าหมายคืนนี้: กดปุ่ม "จำลองเหตุการณ์" แล้วเกิดเอกสารใหม่ใน Firestore และขึ้นในหน้ารายการแจ้งเตือนโดยไม่ต้องรีเฟรชหน้า

เสาร์ 17 ต.ค. ช่วงเช้า: deploy ครั้งแรก

4. GitHub

- `git init`, commit, push เข้า repo กลางใต้โฟลเดอร์ `app/` ตรวจสอบว่าไฟล์ `.env` หรือไฟล์ที่มี API key จริงอยู่ใน `.gitignore` แล้ว ให้ commit แค่ไฟล์ตัวอย่าง (เช่น `firebaseConfig.example.js`) แทน

5. Firebase CLI

```
npm install -g firebase-tools
firebase login
firebase init hosting   # เลือกโปรเจกต์ที่มีอยู่แล้ว, public dir = โฟลเดอร์ของแอป
firebase deploy --only hosting
```

- จะได้ URL สาธารณะแบบ `blimblom.web.app` เปิดในหน้าต่างไม่ระบุตัวตนเพื่อตรวจว่าใช้งานได้โดยไม่ต้องล็อกอิน
- กลับไปที่การจำกัด key ในขั้นตอนที่ 2 เพิ่ม domain นี้เข้าไปถ้ายังไม่ได้ทำ

> เสร็จเมื่อ: URL สาธารณะเปิดแอปได้และแสดงข้อมูลสดจาก Firestore จากเครื่องที่ไม่เคยแตะโน้ตบุ๊กของเราเลย

เสาร์ 17 ต.ค. ช่วงบ่าย/เย็น: push notification + ขัดเกลา

Firebase Cloud Messaging (เขียนในโค้ดแอป)

- ลงทะเบียน service worker (`firebase-messaging-sw.js`) เพื่อรับ push ตอนแอปไม่ได้เปิดอยู่
- ขอสิทธิ์แจ้งเตือนจากผู้ใช้ตอนเปิดแอปครั้งแรก
- ดึง FCM token ของเครื่อง บันทึกลง collection `subscribers` ใน Firestore
- วิธีง่ายที่สุดสำหรับงานแข่งขัน: เมื่อแอปเขียนเหตุการณ์ความเร่งด่วนสูง ให้เรียก FCM REST API ตรงจากฝั่ง client ด้วย token ที่บันทึกไว้เลย ข้ามการใช้ Cloud Functions ไปเพื่อให้ยังไม่มี backend แบบนี้ไม่เหมาะกับโปรดักต์จริง แต่เพียงพอสำหรับเดโม

6. มือถือจริง + Chrome DevTools

- เปิด URL สาธารณะบน Chrome ในมือถือ กด **เพิ่มลงหน้าจอหลัก** แล้วเปิดจากไอคอนนั้นอย่างน้อย 1 ครั้งเพื่อให้นับเป็น PWA ที่ติดตั้งแล้ว
- กดยอมรับ popup ขอสิทธิ์แจ้งเตือน
- จากโน้ตบุ๊ก รันสคริปต์จำลองให้เกิดเหตุการณ์ความเร่งด่วนสูง แล้วตรวจว่า push notification ขึ้นบนมือถือภายในไม่กี่วินาที
- ใช้ Chrome DevTools → แท็บ Application → Service Workers และ Console ช่วยไล่ดูว่าจุดไหนไม่ทำงาน

> หยุดเพิ่มฟีเจอร์คืนนี้ · เสร็จเมื่อ: จำลองเหตุการณ์ช้างแล้ว push notification เด้งขึ้นบนมือถือจริง ครบวงจรบน URL สาธารณะ

อาทิตย์ 18 ต.ค. ช่วงเช้า: ตรวจครั้งสุดท้ายเท่านั้น

- ห้ามเพิ่มฟีเจอร์ใหม่ แก้เฉพาะบั๊กที่เห็นตอนถ่ายวิดีโอเดโม (ทำร่วมกับส่วนวิดีโอ/สไลด์ของทีม)
- เปิด URL สาธารณะบนมือถือเครื่องที่ไม่เคยเปิดมาก่อน ตรวจว่าติดตั้งและรับแจ้งเตือนครั้งแรกยังทำงานได้
- ตรวจว่า Firestore security rules ไม่ได้ปล่อยเป็น test mode โดยไม่รู้ตัว ถ้ายังเปิดอยู่ ยอมรับได้สำหรับเดโมแข่งขัน แต่ถ้ามีคนถามให้บอกว่าเป็นข้อจำกัดที่รู้อยู่แล้ว

ถ้าช้ากว่าแผน – ตัดตามลำดับนี้

1. ปุ่ม รับเรื่อง / กำลังไป / ปิดเหตุ — รายการแจ้งเตือนแบบอ่านอย่างเดียวก็พอ
2. Push notification — แค่หน้ารายการแจ้งเตือนสดก็ยังสื่อไอเดียหลักได้
3. แผนที่หรือชื่อตำแหน่ง — ใช้รายการธรรมดาก็พอ
4. ประเภทเหตุการณ์ที่สอง — เดโมแค่การตรวจจับช้าง ส่วนการตรวจจับเลื่อยยนต์พูดว่า "ออกแบบรองรับไว้แล้ว แต่ไม่ได้โชว์"

ห้ามตัด: การวิเคราะห์ด้วย Gemini ที่ทำงานกับเสียงจริง, URL สาธารณะบน Firebase Hosting, เหตุการณ์อย่างน้อย 1 รายการที่ไหลครบวงจรเข้าไปในหน้ารายการแจ้งเตือน